

RAKSHA BAND

Acoustic AI safety bracelet — project blueprint & technical overview

A slim, self-contained wearable that listens for a scream or a spoken safe-word, confirms it against motion and heart-rate signals, and automatically sends location, alerts contacts, and escalates to emergency services — without requiring a button press.

Prepared for	Hackathon / pilot prototype submission
Form factor	7.2mm thin, adjustable silicone strap
Status	Architecture finalized & hardware integration in progress

1. Purpose & end result

Most personal safety devices fail at the exact moment they're needed: they require a calm, deliberate button press, they depend on a paired smartphone being present and unlocked, and they're not built for a child to operate under stress. Raksha Band removes the requirement for any deliberate action. It listens continuously for a scream or a pre-recorded safe-word, cross-checks that against motion and heart-rate signals to avoid false alarms, and then handles the entire emergency sequence — location, SMS, siren, and escalation to police if nobody responds — on its own.

The end result is a wearable that works fully offline for detection (no cloud dependency for the AI model) and fully independently for communication (its own SIM, not the wearer's phone).

2. What's new in this version

This revision merges two design tracks into one system: an acoustic AI detection layer (real scream classification and custom safe-word recognition, replacing a simple amplitude-threshold microphone) and a communication/escalation layer (GPS location, SMS alerts, and automatic police escalation, which the acoustic-only design didn't address). It also moves the electronics to a single flex-PCB layout so the housing can be built thin enough to wear comfortably all day.

3. System architecture

Every sensor feeds into one onboard controller, which runs both the AI inference and the alert logic. Nothing here depends on a companion phone app to function.

Updated against the actual pin connection sheet: the controller is a standard ESP32-S3 dev board with a separate INMP441 I2S digital microphone, rather than a Sense module's built-in mic — a cleaner audio path for the ML classifier. A physical SOS button and a battery-level ADC input are also part of the real build; see the note below the pin table for how the button fits the design.

Component	Role
ESP32-S3 dev board	Main controller — WiFi/BT, on-device AI inference
INMP441 I2S microphone	Digital audio input for the scream/safe-word classifier
NEO-6M GPS	Live location for the SOS message
SIM800L GSM	SMS + voice call — independent of the wearer's phone
MPU6050 accelerometer	Shake, struggle, and fall detection
MAX30105 pulse sensor	Heart-rate spike as a false-positive filter
Tamper switch	Detects forced strap removal
Buzzer	100dB local siren
Status LED	Visual power/mode indicator
Manual SOS button	Backup fail-safe trigger — see note below

Battery ADC monitor	Reports remaining charge
TP4056 + LiPo	Power, USB-C rechargeable

4. Detection intelligence — two engines

Engine A — scream classification. A 1D convolutional neural network trained via Edge Impulse, running fully on-device. Audio is converted to MFCCs (13 coefficients, 32 filters, 8kHz cutoff) over a 1,000ms sliding window with 500ms stride, classifying ambient audio into scream / silence / background noise.

Engine B — safe-word matching. A Dynamic Time Warping algorithm running alongside the neural network. The wearer records a 3–4 syllable safe-word during setup; the device stores an MFCC template and continuously compares live audio against it. This lets someone customize a discreet trigger phrase without retraining the model or needing a cloud connection.

5. Trigger fusion logic

No single acoustic detection fires an alarm by itself except the two highest-confidence cases (tamper and safe-word). Everything else requires corroboration to filter out false positives.

Signal	Behavior
Tamper loop broken	Fires instantly — no confirmation needed
Safe-word matched (DTW)	Fires instantly — deliberate, high-confidence
Scream classified (ML)	Requires heart-rate spike OR motion signal within a short window
Heart-rate spike alone	Never triggers alone — confirmation signal only
Violent shake / struggle / fall	Fires after a brief confirmation window (0.4–1.2s)
Manual SOS button (GPIO 15)	Fires instantly — backup fail-safe only, never the primary path

The pin sheet includes a manual SOS button (GPIO 15), which is a real, deliberate departure from pure automatic detection worth stating plainly rather than glossing over. It's treated here as a redundant fail-safe layer, not a replacement for the automatic triggers — someone calm enough to press it can, but nothing in the system requires them to. If the intent behind this button is different (e.g. a dedicated setup-mode control), the state machine and messaging in this document should be revisited accordingly.

5b. Pin connection reference

Wiring against the ESP32-S3 dev board, taken directly from the project's connection sheet.

Component	Pin	ESP32-S3 pin	Notes
Status LED	Anode (+)	GPIO 21	220–330Ω series resistor; cathode to GND
Buzzer	Positive (+)	GPIO 3	Negative to GND
Tamper switch	Terminal 1	GPIO 2	Terminal 2 to GND (INPUT_PULLUP)
Manual SOS button	Terminal 1	GPIO 15	Terminal 2 to GND (INPUT_PULLUP)
Battery monitor	Wiper (ADC)	GPIO 1	Potentiometer to simulate voltage; outer legs to 3.3V/GND
SIM800L (GSM)	TX → RX	GPIO 5	ESP receives on GPIO 5
SIM800L (GSM)	RX ← TX	GPIO 4	ESP transmits on GPIO 4
GPS (NEO-6M)	TX → RX	GPIO 7	ESP receives on GPIO 7
GPS (NEO-6M)	RX ← TX	GPIO 6	ESP transmits on GPIO 6
INMP441 mic	WS	GPIO 42	Word select / frame sync
INMP441 mic	SD	GPIO 41	Serial data out
INMP441 mic	SCK	GPIO 43	Serial clock (BCLK)
INMP441 mic	L/R	GND	Mono, left channel only
MPU6050	SDA / SCL	Default I2C	GPIO 8 / GPIO 9
MAX30105	SDA / SCL	Default I2C	Shares the bus with the MPU6050

The MPU6050 and MAX30105 share the I2C bus (default SDA/SCL pins) since both are low-speed sensors that support address-based multiplexing on the same two wires — standard practice, not a shortcut.

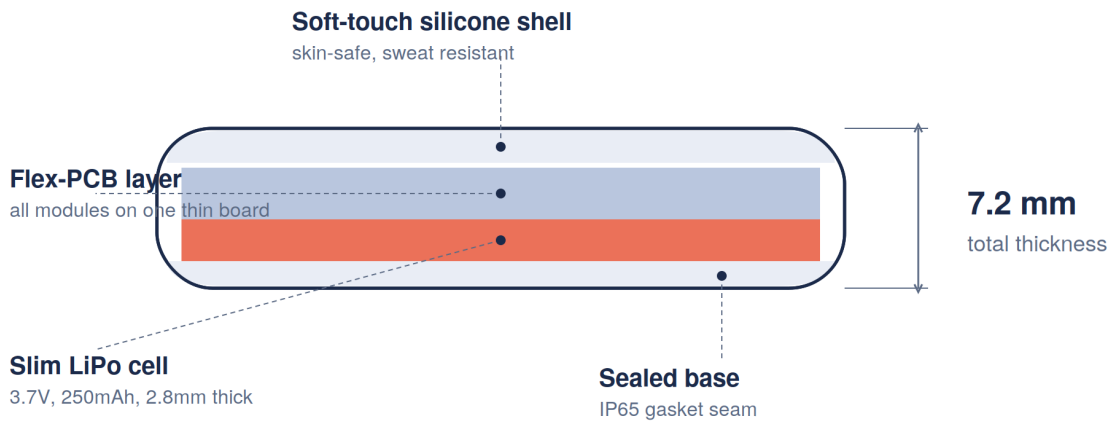
6. Alarm state machine

NORMAL_MODE — passively listening and sensing. **SETUP_MODE** — entered only via a physical button press, used to calmly record a new safe-word beforehand; this is the one legitimate use of a button in the whole system, since it happens outside of any emergency. **ALARM_MODE** — triggered by any of the fused signals above: fetches GPS, sends an SOS SMS with a maps link to emergency contacts, sounds the siren, and resends the location every 45 seconds. A trusted contact can cancel by SMS within 20 seconds; if nobody responds, the device automatically places a voice call to the police as a last resort.

Auto-dialing emergency services from unconfirmed sensor data carries real false-dispatch risk. For a production rollout, this escalation should route through a monitored backend that verifies the alert before contacting police, rather than the device calling emergency services directly.

7. Industrial design — sleek and thin

The earlier prototype spread six separate modules across a wide housing. This version consolidates everything onto a single flex-PCB, stacking the battery beneath it rather than beside it. That's what brings the housing down to a 7.2mm cross-section — thin enough to sit comfortably under a sleeve, closer to a fitness band than a hobbyist enclosure.



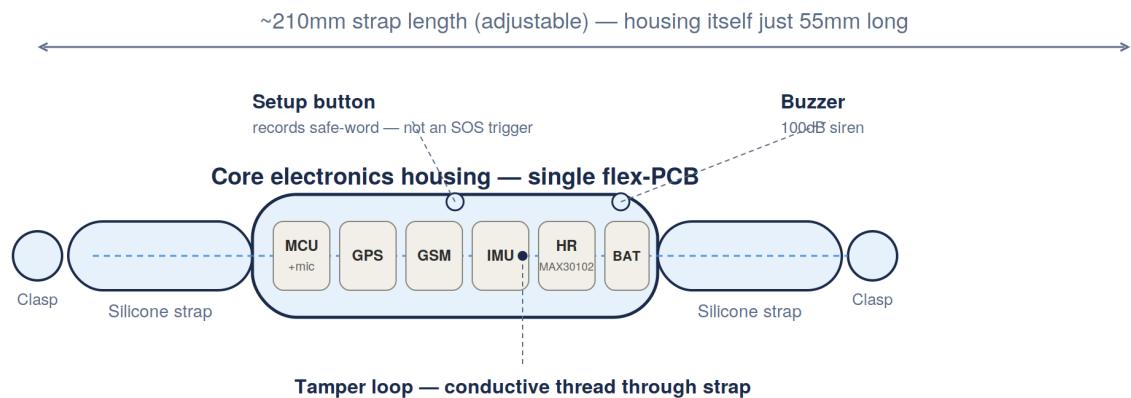
Cross-section through the housing — curved profile follows the wrist

Cross-section of the housing showing the layer stack and overall thickness.

Keeping it this thin comes with real trade-offs worth flagging honestly: a 250mAh cell is smaller than the 500mAh cell in the earlier design, which will shorten standby time, and a single flex-PCB is more expensive to fabricate in small runs than simple breakout modules on a breadboard. Both are reasonable costs for wearability, but they belong in the prototyping budget from the start rather than being a surprise later.

8. Updated component blueprint

Top-down view of the housing with all six modules on the single flex board, plus the tamper loop, buzzer, and setup button.



All six modules sit on one slim flex board — no stacked daughter-boards, which is what keeps the housing thin

All modules share one board — no stacked daughterboards, which is what keeps the housing thin.

9. Prototype bill of materials

Component	Approx. cost
ESP32-S3 dev board	■600–900
INMP441 I2S microphone	■150–250
NEO-6M GPS	■300–450
SIM800L GSM	■350–500
MPU6050	■100
MAX30105 heart-rate sensor	■200–300
Buzzer + tamper switch	■60–100
Status LED + resistor	■5–10
Manual SOS button (backup)	■10
Slim battery + TP4056 charging	■150–250
Flex-PCB + enclosure	■200–350
Total per unit	■2,125–3,120 (~\$25–\$37)

Higher than a bare-microcontroller build — the on-device AI capability, biometric sensing, and flex-PCB construction are the reasons. Cost drops at manufacturing scale, particularly the PCB and enclosure lines.

10. Roadmap

Now	Architecture finalized; Edge Impulse dataset collection underway
Weeks 2–4	Train and validate scream classifier (target 85%+ accuracy); implement DTW matching
Weeks 4–6	Breadboard integration of all sensors; sensor-fusion logic gate testing
3 months	Flex-PCB revision, enclosure prototyping, pilot with 20–30 users
6–12 months	Manufacturing partner; monitored-backend escalation layer for safe police handoff

Protection that doesn't wait for permission — no button, no app, no hesitation.